

Tema 3. Validez de Documentos.

Como ya se ha expuesto, un documento XML correcto es aquel que cumple dos condiciones:

- Ser un documento bien formado.
- Ser un documento válido.

En este tema se explicará la Definición de Tipo de Documento (o DTD) y los XML Schemas (o Esquemas XML).

Las diferencias principales entre las DTD y los Schemas son:

- ✓ Las DTD tienen una sintaxis específica mientras que los Schema utilizan sintaxis XML.
- ✓ Un Schema se puede manipular, como cualquier otro documento XML.
- ✓ Hay muchas más herramientas para trabajar con DTD que con Schema.
- ✓ Un Schema soporta la integración de los espacios de nombres permitiendo asociar nodos de un documento con declaraciones de tipo de un esquema.
- ✓ La DTD sólo permite una asociación entre un documento y su DTD.

Definición de Tipo de Documento (Document Type Definition).

INTRODUCCIÓN

Como se mencionó en el tema anterior, es posible asociar un documento XML a una DTD determinado para así organizar la información contenida de forma que nos sea más útil de cara a nuestro objetivo. Sin embargo, esto no es imprescindible ya que puede haber documentos XML cuya estructura no esté definida en ningún DTD.

Por tanto, atendiendo al hecho de que un documento lleve o no asociado una DTD, podemos diferenciar dos tipos de documentos XML:

- Documentos bien formados son aquellos que no tienen asociado una DTD, pero cumplen todas las reglas de XML.
- Documentos válidos son aquellos que siguen las reglas de una DTD específica.

Evidentemente, un documento *válido* estará siempre *bien formado*.

Si queremos tener la seguridad de que un fichero XML pueda ser perfectamente interpretado por cualquier herramienta, podemos incluir una definición de su construcción que preceda a los datos propiamente dichos. Este conjunto de meta-datos es el que recibe el nombre de DTD.

Las DTD son la clave de auto-descripción en documentos XML, es decir, permiten definir al usuario qué significa exactamente cada una de las marcas que va a incluir a continuación para identificar los datos.

En definitiva, la definición de tipo de documento o DTD contiene las reglas sintácticas que deben cumplir los documentos de un tipo determinado

Un documento XML indica al procesador XML que va a utilizar algún tipo de definición de tipo de documento por medio de la línea de declaración de tipo de documento `<!DOCTYPE...>` con la cual se le indica que lea las definiciones de la DTD y compruebe que el documento XML cumple con las normas estructurales y sintácticas establecidas.

A este proceso se le conoce con el nombre de validación y a los documentos que superan con éxito la prueba de validación se les denomina documentos XML válidos y aquellos que no cumplan las normas establecidas en la DTD son conocidos como documentos XML no válidos.

EJEMPLO DE CÓMO DEFINIR UNA DTD

Una DTD es un documento en el cual se declaran entidades, notaciones, elementos y lista de atributos permitidos en aquellos documentos XML que la utilicen.

Por lo tanto, la DTD es un mecanismo para describir los objetos que pueden aparecer en un documento XML.

XML

Validez de Documentos

Antes de explicar cómo se declara una entidad, una notación, un elemento o lista de atributos mostraremos un ejemplo.

A continuación se muestra la solución XML que describe a una persona.

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<persona>
  <nombre> Juan </nombre>
  <apellido> Pérez </apellido>
  <profesion> Taxista </profesion>
</persona>
```

Como una DTD sólo describe el tipo general, no la instancia específica, la DTD para este ejemplo sería un elemento persona que contiene unos elementos hijos nombre, apellido y profesion. Cada uno de estos elementos contiene un texto. Por tanto, la DTD sería:

```
<!ELEMENT persona (nombre, apellido, profesion)>
<!ELEMENT nombre (#PCDATA)>
<!ELEMENT apellido (#PCDATA)>
<!ELEMENT profesion (#PCDATA)>
```

Esta DTD se puede guardar:

1. En un archivo independiente del documento XML: de esta forma diferentes ficheros XML pueden referenciarlo si es necesario.

Dentro del documento XML se haría una referencia al fichero .dtd que contendría las definiciones utilizando una declaración de tipo de documento.

Una declaración de tipo de documento es:

```
<!DOCTYPE nombre_elemento_raiz SYSTEM referencia_fichero_dtd>
```

Por ejemplo:

```
<!DOCTYPE persona SYSTEM "persona.dtd">
```

La declaración del tipo de documento se incluye en el prólogo del documento XML, después de la declaración XML, antes del elemento raíz.

En el ejemplo:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE persona SYSTEM "persona.dtd">
<persona>
  <nombre> Juan </nombre>
  <apellido> Pérez </apellido>
  <profesion> Taxista </profesion>
</persona>
```

Existiendo el fichero "persona.dtd" en el mismo directorio que el documento XML, sino habría que utilizar una ruta relativa hasta llegar a él.

Código fuente de persona.dtd:

```
<!ELEMENT persona (nombre, apellido, profesion)>
<!ELEMENT nombre (#PCDATA)>
<!ELEMENT apellido (#PCDATA)>
<!ELEMENT profesion (#PCDATA)>
```

2. O dentro del propio documento XML, usando la declaración de tipo utilizada anteriormente.

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE persona[
  <!ELEMENT persona (nombre, apellido, profesion)>
  <!ELEMENT nombre (#PCDATA)>
  <!ELEMENT apellido (#PCDATA)>
  <!ELEMENT profesion (#PCDATA)>
]>
<persona>
  <nombre> Juan </nombre>
  <apellido> Pérez </apellido>
  <profesion> Taxista </profesion>
</persona>
```

DECLARACIÓN DE ELEMENTOS

Todos los elementos usados en un documento válido, deben declararse en la DTD de un documento con una declaración de elemento.

La sintaxis genérica es:

```
<!ELEMENT nombre_elemento especificación_de_contenido>
```

El *nombre_elemento* se corresponde al nombre de la etiqueta que hemos puesto para dicho elemento.

La *especificación_de_contenido* indica el hijo que puede o debe tener y su orden. Estas especificaciones pueden ser bastante complejas: pueden establecer que un elemento debe tener tres elementos hijos de un tipo determinado o dos hijos de un tipo seguidos por otro elemento de otro tipo, etc.

A continuación, se explicarán las posibilidades existentes en la declaración de elementos.

- **#PCDATA, EMPTY, ANY**

- *#PCDATA*: Indica que un elemento puede contener sólo datos de caracteres, pero que no puede contener ningún elemento hijo para ningún tipo.

Por ejemplo, la declaración del elemento nombre indica que puede contener texto pero no puede contener elementos.

```
<!ELEMENT nombre (#PCDATA)>
```

- *EMPTY*: indica que el elemento es un elemento vacío

Un elemento vacío es un elemento que no tiene asociado contenido.

Por ejemplo:

```
<imagen src="foto.jpg"/>
```

La declaración de un elemento de este tipo sería:

```
<!ELEMENT imagen EMPTY>
```

- *ANY*: indica que el elemento puede contener cualquier otro elemento declarado en la DTD. Raramente se utiliza

▪ Elementos hijos:

- Un único hijo

Cuando un elemento contiene exactamente un elemento hijo de un tipo determinado.

En este caso, la especificación de contenido consta del nombre del elemento hijo entre paréntesis.

```
<!ELEMENT nombre_elemento (nombre_elemento_hijo)>
```

- Otras cantidades de hijos

La mayoría de los elementos contienen datos de caracteres o múltiples elementos hijos. En este último caso, los hijos se especifican separándolos por comas (es decir, utilizando lo que se denominan secuencias). La secuencia indica que los elementos especificados deben aparecer en el orden establecido.

Por ejemplo: Supongamos que tenemos un elemento nombre_persona definido por tres elementos hijos: nombre, apellido y mote.

```
<nombre_persona>  
  <nombre> Francisco </nombre>  
  <apellido> Ramirez </apellido>  
  <mote> Paquito </mote>  
</nombre_persona>
```

La declaración del elemento nombre_persona sería:

```
<!ELEMENT nombre_persona (nombre, apellido, mote)>
```

No todas las instancias de elementos tienen porque tener valor para los mismos elementos hijos.

Podemos fijar uno o tres sufijos a un nombre de elemento en una especificación de contenido para indicar qué cantidad de hijos de dicho elemento se esperan en dicha posición:

Sufijo ?: admite 0 ó 1 hijo de elemento

Sufijo *: admite 0 ó más hijos de elementos.

Sufijo +: admite 1 o más hijos de elementos.

Por ejemplo: La declaración de elemento podría ser que el elemento nombre_persona tiene que contener exactamente un nombre y apellido; y puede o no contener un mote.

```
<!ELEMENT nombre_persona (nombre, apellido, mote?)>
```

▪ Opciones

¿Y qué sucede si un elemento tiene elementos hijo diferentes? Es decir, una instancia de un objeto puede contener un tipo de hijo y otra instancia diferente puede tener otro hijo de diferente tipo.

La solución para estos casos es una lista de nombres de elemento separados por barras verticales.

Por ejemplo: En una tienda en la que se alquilan películas o videojuegos tendríamos:

```
<!ELEMENT producto (pelicula | videojuego)>
```

Es importante no olvidar que el producto será una película o un videojuego, no podrá ser ambas cosas a la vez.

▪ Paréntesis

Todas las opciones anteriores se pueden combinar usando paréntesis.

Por ejemplo: definimos un elemento círculo con un elemento centro y con otro elemento que puede ser el radio o diámetro del círculo.

```
<!ELEMENT circulo (centro , ( radio| diametro))>
```

DECLARACIÓN DE LISTA DE ATRIBUTOS

Un documento válido también debe declarar todos los atributos de los elementos. Para ello, se harán declaraciones *ATTLIST*.

Una sola declaración *ATTLIST* puede declarar múltiples atributos para un solo tipo de elemento. Sin embargo, si el mismo atributo se repite en múltiples elementos, debe declararse independientemente para cada elemento donde aparece.

La sintaxis general para declarar los atributos es:

```
<!ATTLIST nombre_elemento nombre_atributo tipo_atributo propiedades>
```

En nombre_elemento se especificará el nombre de la etiqueta correspondiente al elemento.

En nombre_atributo se especificará el nombre definido para el atributo en cuestión.

En tipo de atributo puede ser:

- CDATA (Character DATA): un valor de este tipo puede contener cualquier cadena de texto aceptable en una buena estructura XML.

Ejemplo:

```
<!ATTLIST imagen texto_alternativo CDATA #IMPLIED>
```

- NMTOKEN (NaMe Token): un valor de este tipo será una cadena de caracteres, pudiendo contener letras minúsculas, letras mayúsculas, números, puntos ".", guiones medios "-", guiones bajos "_" o el carácter dos puntos ":".

Ejemplo:

```
<!ATTLIST dia anio NMTOKEN #REQUIRED>
```

- Enumeración: es el único tipo de atributo que no es una palabra clave XML, sino una lista con los valores posibles para dicho atributo.

Ejemplo:

```
<!ATTLIST fecha mes (1|2|3|4|5|6|7|8|9|10|11|12) #REQUIRED>
```

- ENTITY: contiene el nombre de una entidad sin analizar, declarada en alguna parte en la DTD.

Ejemplo:

```
<!ATTLIST pelicula origen ENTITY #REQUIRED>
```

```
<pelicula origen="presentacion.avi" />
```

- ENTITIES: contiene nombres de una o más entidades sin analizar, declaradas en cualquier parte en la DTD, separados por un espacio en blanco.

Ejemplo:

```
<!ATTLIST grupos imagenes ENTITIES #REQUIRED>
```

XML

Validez de Documentos

- ID: contiene un nombre XML, único dentro del documento XML. Ningún otro atributo de tipo ID puede tener el mismo valor en el documento.

Ejemplo:

```
<!ATTLIST persona dni ID #REQUIRED>
```

- IDREF: se refiere al atributo de tipo ID de algún elemento en el documento. Por lo tanto, debe ser un nombre XML. Normalmente, se usan para establecer relaciones entre los elementos.

Ejemplo:

```
<!ATTLIST pelicula direccion IDREF #REQUIRED>
```

- IDREFS: contiene una lista de nombres XML separados por un espacio en blanco, cada uno de los cuales debe tener un ID de un elemento del documento.

Ejemplo:

```
<!ATTLIST director filmografia IDREFS #REQUIRED>
```

```
<director filmografia="P1 P3 P4">James Cameron</director>
```

- NOTATION: contiene el nombre de una notación declarada en la DTD del documento. Tal vez sea el atributo menos usado.

```
<!ATTLIST animal
```

```
imagen CDATA #IMPLIED
```

```
tipo_de_imagen NOTATION (jpg | gif | png) #IMPLIED>
```

Indica que los elementos "animal" que se escriban, deben incluir opcionalmente el atributo tipo_de_imagen, cuyo valor será una notación (gif, jpg o png).

Las propiedades pueden ser:

- #IMPLIED: significa que el atributo es opcional. Es decir, cada instancia de dicho elemento puede o no tener un valor para el atributo.
- #REQUIRED: significa que el atributo es necesario. Es decir, cada instancia debe proporcionar un valor para el atributo de ese elemento.
- #FIXED "valor": significa que el valor del atributo es constante.
- Literal: el valor predeterminado real se proporciona como cadena entre comillas dobles. Valor por defecto.

DECLARACIÓN DE ENTIDADES

Las entidades representan contenidos y referencias de texto, existen dos formas de definir las:

- `<!ENTITY nombre "valor">`
- `<!ENTITY nombre SYSTEM "valor">`

Clasificación de entidades:

- *Interna* (a un documento) y *Externa* (accesibles URI)
- *General* (& en documento XML) y *Parámetro* (%) en DTD)

Entidades predefinidas:

& - < - > - " - '

Entidades internas:

`<!ENTITY copyright "Copyright IES LC">`

Entidades externas:

`<!ENTITY copyright SYSTEM "copy.dat">`

Espacios de Nombre.

En XML se llama espacio de nombres a una colección de nombres que proporciona un mecanismo por el que los nombres de elementos y atributos pueden asignarse para diferentes usos dependiendo de cada caso, utilizando los prefijos adecuados. Es decir, permiten definir la pertenencia de los elementos y atributos de un elemento XML a una familia de vocabulario XML.

En XML, los espacios de nombre tienen dos propósitos:

- Distinguir entre elementos y atributos con distintos significados, pero que tienen el mismo nombre.
- Agrupar todos los elementos y atributos relacionados de una sola aplicación XML para que el software pueda reconocerlos con facilidad.

Deben cargarse previamente al principio del documento XML antes de poder utilizar los prefijos.

`<nombre_elemento xmlns:alias="URI">`

donde URI es un hipervínculo de acceso a un recurso remoto

Gracias a los espacios de nombres, es posible integrar etiquetas de diversos lenguajes en un mismo documento XML.

Para que resulte fácil su aplicación vamos a imaginar un supuesto:

```
<pais nombre="España">  
<capital>Madrid</capital>  
</pais>
```

```
<inversion>  
<capital>700000€</capital>  
</inversion>
```

¿Cómo se combinarían los ejemplos anteriores en un mismo documento? ¿Cómo distinguiríamos el capital cuando nos referimos a la capital de un país determinado del capital de una inversión económica?

```
<ejemplo>  
<inversiones>  
  <país nombre="España">  
    <capital>Madrid</capital>  
    <capital>700000€</capital>  
  </país>  
  ...  
</inversiones>  
</ejemplo>
```

```
<ejemplo xmlns:bolsa="http://www.bolsa.com" xmlns:geog="http://www.geografia.es">
  <bolsa:inversiones>

    <geog:país geog:nombre="España">
      <geog:capital>Madrid</geog:capital>
      <bolsa:capital>700000€</bolsa:capital>
    </geog:país>
    . . .
  </bolsa:inversiones>
</ejemplo>
```

XML Schemas.

INTRODUCCIÓN

Antes de comenzar a exponer las diferencias entre las DTDs y los XML Schemas es importante revisar de dónde y cómo surgieron las dos principales soluciones en las que nos hemos detenido.

Inicialmente el uso de las DTDs se debe a SGML, en el cual describían no sólo el vocabulario necesario para identificar todos los elementos de que iba a constar nuestro documento, sino que también expresaba la estructura que dichos elementos debía respetar.

Con la creación de XML, que no es más que un subconjunto de SGML, fue necesario mantener la posibilidad de describir los elementos necesarios, al igual que en SGML, por ello se importaron las DTDs y todo su comportamiento.

Posteriormente, y debido al uso principalmente, se vio la necesidad de emplear otros métodos para describir esas necesidades inherentes a XML, con esta idea se crearon los XML Schemas. Con ellos se pretendía mejorar y ampliar la utilidad de las DTDs.

Un Esquema XML o XML Schema es un documento que contiene una descripción formal de lo que comprende un documento XML válido. Sería una definición de la estructura de un conjunto de documentos XML. Son similares a los DTDs en el sentido que definen qué elementos puede contener un documento XML, cómo están organizados, y qué atributos y de qué tipo pueden tener sus elementos.

Un documento XML se describe mediante un esquema denominado documento de instancia. Si un documento satisface todas las restricciones especificadas por el esquema, se considera que es un documento con un esquema válido. El documento del esquema se asocia con un documento de instancia a través de uno de los siguientes métodos:

- Atributo *xsi:schemaLocation* en un elemento contiene una lista de espacios de nombre usados dentro de dicho elemento y los URL de los esquemas con los que validan los elementos y atributos en dichos espacios de nombre.
- Atributo *xsi:noNamespaceSchemaLocation* contiene una URL para el esquema usado para validar elementos que no se encuentran en ningún espacio de nombres.

ESQUEMAS Y DTD

Originalmente se utilizaron los DTDs, posteriormente se han desarrollado los XML Schemas.

Las DTD proporcionan la capacidad de realizar una validación básica de los siguientes elementos:

- Anidamiento de elementos.
- Limitaciones de ocurrencia de elementos.
- Atributos permitidos.
- Tipos de atributos y valores predeterminados.

Sin embargo, las DTD no proporcionan un buen control sobre el formato y tipos de dato del elemento y valores de atributos. En los tipos de atributo distintos a los especiales (ID, IDREF, ENTITY...) cuando un elemento o atributo se ha declarado para contener datos de caracteres, no se deben incluir límites sobre la longitud, el tipo o el formato del contenido.

Para documentos narrativos, como libros o páginas Web, este nivel control probablemente sea suficiente. Pero, para el caso de llamadas a procedimientos remotos o serialización de objetos, es importante tener un control preciso sobre el contenido del texto de los elementos y atributos.

Los DTDs tienen las siguientes limitaciones:

- Su sintaxis, al no ser XML, son difíciles de manipular.
- No soportan los espacios de nombres.
- No permiten especificar tipos de datos (por ejemplo: enteros, flotantes, fechas, etc.)
- No permiten especificar secuencias no ordenadas como ((e1,e2,e3)|(e1,e3,e2)|(e2,e1,e3)|...(e3,e2,e1)).
- No hay soporte para declaraciones sensibles al contexto. Los elementos se definen todos a nivel de documento, ejemplo, contenido con el mismo nombre cuya estructura cambia en diferentes contextos.
- Soporte limitado para referencias cruzadas, no es posible formar claves a partir de varios atributos o de elementos.
- No son extensibles (una vez definido, no es posible añadir nuevos vocabularios a un DTD).

En este sentido, el estándar de esquemas XML incluye las siguientes particularidades:

- Sintaxis XML.
- Soporte para espacios de nombres.
- Gran cantidad de tipos de datos predefinidos y creación de tipos de datos por el usuario.

- Extensibilidad: Inclusión/Redefinición de esquemas y herencia de tipos de datos.

ELEMENTOS BÁSICOS DE LOS ESQUEMAS

Permite especificar rangos de inclusión:

```
<xs:element name="alumno" minOccurs="1" maxOccurs="200"
            type="TipoAlumno"/>
```

Permite también especificar tipos de datos:

```
<xs:element name="nacim" type="xs:gYear"/>
```

En los XML se pueden utilizar Tipos anónimos y con nombre:

Tipos anónimos (código más legible)

```
<xs:element name="alumno">
  <xs:sequence>
    <xs:element name="nombre" type="xs:string"/>
    <xs:element name="apellidos" type="xs:string"/>
  </xs:sequence>
</xs:element>
```

Tipos con nombre (código más reutilizable)

```
...
<xs:element name="alumno" type="TipoAlumno"/>
...
<xs:ComplexType name="TipoAlumno">
  <xs:sequence>
    <xs:element name="nombre" type="xs:string"/>
    <xs:element name="apellidos" type="xs:string"/>
  </xs:sequence>
</xs:ComplexType>
```

En los XML Schema es posible nombrar agrupaciones de elementos y de atributos para hacer referencias a ellas.

```
<xs:group name="nombApell">
  <xs:sequence>
    <xs:element name="nombre" type="xs:string"/>
    <xs:element name="apellidos" type="xs:string"/>
  </xs:sequence>
</xs:group>
```

```
<xs:complexType type="TipoAlumno">
  <xs:group ref=" nombApell" />
  <xs:element name="curso" type="xs:string"/>
</xs:complexType>
```

Los tipos complejos son aquellos que pueden contener elementos o atributos. Su construcción básica se realiza mediante una enumeración de elementos.

```
<xs:complexType name="TipoAlumno">
  <xs:sequence>
    <xs:element name="nombre" type="xs:string"/>
    <xs:element name="apellidos" type="xs:string"/>
    <xs:element name="nacim" type="xs:gYear"
      minOccurs="0" maxOccurs="1"/>
  </xs:sequence>
  <xs:attribute name="dni" type="xs:integer"/>
</xs:complexType>
```



```
<alumno dni="53032765">
  <nombre>Lorenzo</nombre>
  <apellidos>Fernandez Gil</apellidos>
  <nacim>1979</nacim>
</alumno>
```

Una alternativa es utilizar Choice, que conlleva una o exclusiva.

```
<xs:complexType name="Transporte">
  <xs:choice>
    <xs:element name="autobus" type="xs:string"/>
    <xs:element name="tren" type="xs:string"/>
    <xs:element name="avion" type="xs:string"/>
  </xs:choice>
</xs:complexType>
```



```
<transporte>
<tren>Talgo 770</tren>
</transporte>
```

Trabajando con tipos complejos, el contenido mixto permite mezclar texto con elementos.

```
<xs:complexType name="TCom" mixed="true">
  <xs:choice minOccurs="0" maxOccurs="unbounded">
    <xs:element name="emph" type="xs:string"/>
  </xs:choice>
</xs:complexType>

<xs:element name="comentarios" type="TCom" />
```



```
<comentarios>
Se distrae con <emph>mucha facilidad</emph>
</comentarios>
```

Para secuencias no ordenadas se utiliza "all" para que aparezcan todos los elementos en cualquier orden.

```
<xs:complexType name="TipoLibro">
  <xs:all>
    <xs:element name="autor" type="xs:string"/>
    <xs:element name="titulo" type="xs:string"/>
  </xs:all>
</xs:complexType>
<xs:element name="libro" type="TipoLibro" />
```

```
<libro>
<autor>Nuria Roca</autor>
<título>Los caracoles no saben que son
caracoles</título>
</libro>
```

```
<libro>
<título>La sombra del viento</título>
<autor>Carlos Ruiz Zafon</autor>
</libro>
```

En los XML Schema, los Tipos simples no pueden contener elementos o atributos. Pueden ser:

- Predefinidos (o también llamados built-in o definidos en la especificación):
 - ➔ Primitivos: string, boolean, Lumber, flota, double, duration, dateTime, time, date, gYearMonth, gYear, gMonthDay, gDay, gMonth, hexBinary, base64Binary, anyURI, QName (nombre cualificado con espacio de nombres) o Notation (notación binaria).
 - ➔ Derivados: normalizedString, token, language, IDRefs, Entities, NMTOKEN, NMTOKENS, Name, NCName, ID, IDREF, Entity, integer, nonPositiveInteger,

XML

negativeInteger, log, int, short, byte, nonNegativeInteger, unsignedLong, unsignedInt, unsignedShort, unsignedByte, positiveInteger.

- Definidos por el usuario a partir de tipos predefinidos.

En los Esquemas XML, dependiendo de la relación entre los valores se puede utilizar:

- Equal: Igualdad entre valores de un tipo de datos.
- Ordered: Relaciones de orden entre valores.
- Bounded: Límites inferiores y superiores para valores.
- Cardinality: Define si es finito o infinito (contable o no contable).

Y para limitar valores y expresiones:

- Length, minlength o maxlength: Longitudes del tipo de datos.
- Pattern: Restricciones sobre valores mediante expresiones regulares.
- Enumeration: Restringe a una determinada enumeración de valores.
- Whitespaces: Define la política de tratamiento de espacios (preserve / replace / collapse).
- (max/min)(in/ex)clusive: Límites superiores/inferiores del tipo de datos.
- totalDigits, fractionDigits: número de dígitos totales y decimales.

Veamos a continuación ejemplos en los que se emplea enumeration y restriction:

Enumeration

```
<xs:simpleType name="TipoCarrera">
<xs:restriction base="xs:token">
  <xs:enumeration value="Informatica de Gestion"/>
  <xs:enumeration value="Informatica de Sistemas"/>
</xs:restriction>
</xs:simpleType>
```

Restriction

```
<xs:simpleType name="mes">
<xs:restriction base="xs:integer">
  <xs:minInclusive value="1" />
  <xs:maxInclusive value="31" />
</xs:restriction>
```

```
</xs:simpleType>
```

Veamos un ejemplo de listas con Restriction

```
<xs:simpleType name="ComponentesRGB">
  <xs:list itemType="ComponenteRGB"/>
</xs:simpleType>

<xs:simpleType name="ComponenteRGB">
<xs:restriction base="xs:nonNegativeInteger">
  <xs:maxInclusive value="255" />
</xs:restriction>
</xs:simpleType>
```

Se puede aplicar length, maxLength y minLength

```
<xs:simpleType name="ColorRGB">
  <xs:restriction base="ComponentesRGB">
    <xs:length value="3" />
  </xs:restriction>
</xs:simpleType>
```



```
<color>255 255 0</color>
```

Y a continuación veamos un ejemplo de Uniones:

```
<xs:simpleType name="TipoNota">
<xs:union>
  <xs:simpleType>
    <xs:restriction base="xs:float">
      <xs:maxInclusive value="10" />
      <xs:minInclusive value="0" />
    </xs:restriction>
  </xs:simpleType>
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:enumeration value="No presentado" />
    </xs:restriction>
  </xs:simpleType>
</xs:union>
</xs:simpleType>
<xs:element name="nota" type="TipoNota" />
```

<nota>8.50</nota>
<nota>No presentado</nota>

Pueden aparecer las siguientes expresiones regulares:

Expresión	Posibles valores
Elemento /d	Elemento 2
a*b	b, ab, aab, aaab,...
[xyz]b	xb,yb,zb
a?b	b, ab
a+b	ab, aab, aaab,...
[a-c]x	ax, bx, cx
[^0-9]x	Carácter != dígito seguido de x
\Dx	Carácter != dígito seguido de x
.abc	Cualquier carácter seguido de abc
\n	Salto de línea

Veamos un ejemplo:

```
<xs:simpleType name="NIF">
  <xs:restriction base="xs:token">
    <xs:pattern value="\d{7,8}[A-Z]" />
  </xs:restriction>
</xs:simpleType>
<xs:element name="nif" type="NIF" />
```



<nif>9003664S</nif>
<nota>11079846X</nota>

También podemos encontrarnos tipos derivados por extensión, sería similar a las subclases de Programación Orientada a Objetos, que consiste en añadir elementos a un tipo base.

```
<xs:complexType name="Figura" >
  <xs:attribute name="color" type="Color"/>
</xs:complexType>

<xs:complexType name="Rectangulo">
  <xs:complexContent>
    <xs:extension base="Figura">
      <xs:attribute name="base" type="xs:float" />
      <xs:attribute name="altura" type="xs:float" />
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

<xs:complexType name="Circulo">
```

```
...habría que incluir el radio
</xs:complexType>
```

Los tipos derivados pueden utilizarse donde se utiliza la clase base:

```
<xs:element name="figuras">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="figura" type="Figura"
        maxOccurs="unbounded" />
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

```
<figuras>
  <figura base="23" altura="3" xsi:type="Rectangulo" />
  <figura radio="3" xsi:type="Circulo" />
</figuras>
```

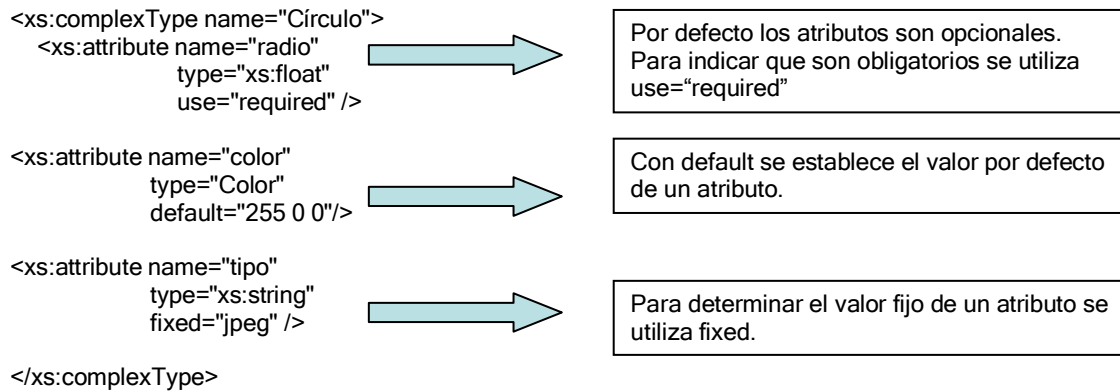


Es necesario especificar el tipo mediante `xsi:type`

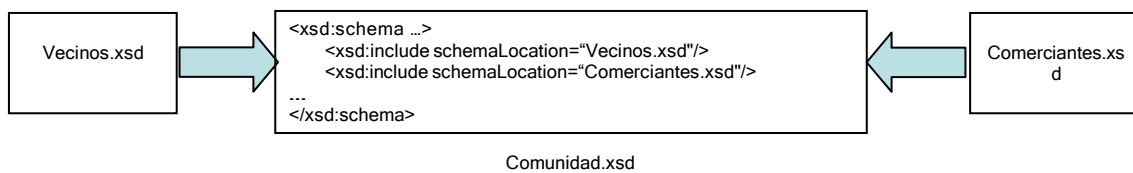
En los XML Schema también pueden aparecer tipos abstractos. Se declaran mediante `abstract="true"`. No puede usarse directamente:

```
<xs:complexType name="Figura" abstract="true">
  <xs:attribute name="color" type="Color"/>
</xs:complexType>
```

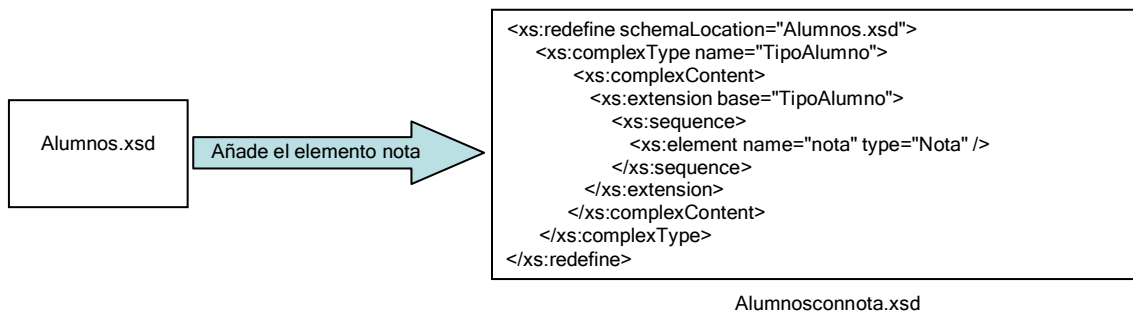
Llegados a este punto nos detendremos en la declaración de atributos:



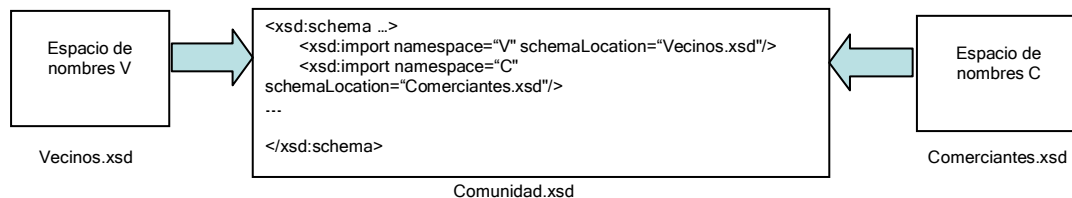
Para incluir elementos de otros esquemas se emplea *include*. Los elementos deben estar en el mismo espacio de nombres.



Redefine es similar a *include*; pero permite modificar los elementos incluidos:



Import permite incluir elementos de otros esquemas con distintos espacios de nombres:



Para finalizar, sobre las claves y la unicidad en los XML Schema se comentará lo siguiente:

- Los DTDs proporcionan el atributo ID para marcar la unicidad (un valor ID era único en todo el documento).
- Los XML Schema tienen más posibilidades:
 - ➔ Indicar que un elemento es único (con unique).
 - ➔ Definir atributos únicos.
 - ➔ Definir combinaciones de elementos y atributos como únicos.
 - ➔ Distinguir entre unicidad y claves (con key). Además de ser único, debe existir y no puede ser nulo.

Mediante una serie de instrucciones se va a poder simular el comportamiento de las restricciones de clave primaria, de clave ajena y de unicidad, que permiten preservar la integridad referencial en los sistemas gestores de bases de datos.

key

Permite definir un elemento o atributo como clave. El valor de la clave no puede ser nulo ni repetirse, de manera que identifica de manera única al elemento/atributo al que se asocia.

```
<xs:key name='.....'>
  <xs:selector xpath='....' />
  <xs:field xpath='.....' />
</xs:key>
```

Ejemplo:

```
<xs:key name='prodNumKey'> -- Ponemos un nombre a la clave para poder después hacer
referencia a ella.
```

```
<xs:selector xpath='productos/producto' /> -- Indicamos donde está posicionado el elemento que va
a ser clave primaria.
```

```
<xs:field xpath='identificador' /> --Indicamos el elemento que contiene el valor de la clave primaria.
```

keyref

Permite referenciar a claves primarias (xs:key) o claves únicas (xs:unique), compuestas por elemento y/o atributos.

Es un comportamiento equivalente a una clave ajena en una tabla de un sistema gestor de bases de datos relacional.

```
<xs:keyref name='.....' refer='...'>
  <xs:selector xpath='....' />
  <xs:field xpath='.....' />
</xs:keyref>
```

Ejemplo:

```
<xs:keyref name='proNumKeyRef' refer='prodNumKey'> -- Hacemos referencia a la clave primaria a
través del atributo refer.
```

```
<xs:selector xpath='lineas/linea' /> -- Indicamos donde está posicionado
```

```
<xs:field xpath='@identificador' /> --Cuando se hace referencia a un atributo se pone @
```

unique

Permite definir un elemento o atributo como de valor único. Es un comportamiento equivalente a una clave única en una tabla de un sistema gestor de bases de datos relacional.

Esta instrucción se declara de manera conjunta con `xs:keyref`, que permitirá a otro elemento o atributo referenciar a la clave única.

```
<xs:unique name='.....'>
<xs:selector xpath='.....'/>
<xs:field xpath='.....'/>
</xs:unique>
```

Fichero ejemplo XML:

```
.....
<productos>
  <producto>
    <identificador>55</identificador>
    <nombre>Pantalón</nombre>
    .....
  </producto>
  <producto>
    <identificador>32</identificador>
    <nombre>Abrigo</nombre>
    .....
  </producto>
  .....
</productos>
<lineas>
  <línea identificador="55">
    <cantidad>2</cantidad>
    .....
  </línea>
  <línea identificador="32">
    <cantidad>2</cantidad>
    .....
  </línea>
  .....
</lineas>
```